

ProblemHunt

Real problems. Tested solutions.

Product transformation plan | Community knowledge, reputation, voluntary tips and future grounded AI | 25 September 2026

ProblemHunt is a technical problem-solving community. People publish real problems, builders propose solutions, the author tests them, and the fix that worked becomes part of a searchable knowledge library. Participation is free. Builders earn category-specific reputation and may receive optional tips through an external payment link.

The product promise

ProblemHunt helps someone move from a confusing symptom to a tested fix. It values evidence over confident guesses: what was observed, what was attempted, what changed, and how the result was verified.

Initial audience

Launch with two domains where the founder can understand the content, seed authentic examples and recruit early contributors.

Cloud Computing and DevOps	Professional AV and Audio
AWS, Azure, Linux, networking, Docker, Terraform, CI/CD and deployment	Audio consoles, RF, signal flow, video, projection, conferencing, control and cabling

Core loop

- **Post:** a user describes the symptom, environment, expected result, actual result and prior attempts.
- **Propose:** builders submit diagnoses and step-by-step solutions with reasoning, verification and sources.
- **Test:** the author tries a solution and reports the observation.
- **Resolve:** the author accepts the solution that worked and records why it worked.
- **Reuse:** the solved case becomes searchable; the community can upvote useful answers.
- **Reward:** the builder gains reputation and can optionally receive a tip outside the platform.

Success definition

The first release succeeds when a visitor can find a solved case, apply it safely, and understand why it worked - or post a well-structured new problem when no match exists.

Transformation of the current application

Reuse the working foundation while changing the product model.

Current concept	New ProblemHunt concept	Action
Brief	Technical problem	Rename and expand structured fields
Proposal	Proposed solution	Keep the relationship; change language and validation
Client	Problem author	Use ordinary community terminology
Builder	Problem hunter / contributor	Keep profile, add expertise and reputation
Accepted proposal	Accepted solution	Represents a fix tested by the author
Job status	Open, Testing, Solved, Closed	Replace marketplace states
Transaction / wallet	Optional external tip link	Remove custody and transaction logic
Payment dispute	Content report and moderation	Replace financial dispute workflow

Remove

- Deposits, escrow language, balances, wallets, payout releases and refund states.
- Promises that ProblemHunt guarantees work, controls funds or arbitrates commercial contracts.
- Marketplace language that makes every interaction feel like a paid job.

Add

- Domains, categories, tags, product/model/version metadata and structured diagnostic context.
- Solution comments for clarification, solution upvotes and one author-accepted answer.
- Solved-case pages, category search, contributor profiles and reputation event history.
- Reporting, moderation, duplicate detection, content revision history and privacy controls.
- An optional external tip URL on contributor profiles. Tips never affect rank or acceptance.

Content states

State	Meaning
Open	The author is seeking possible diagnoses or fixes.
Testing	The author is actively trying one or more proposed solutions.
Solved	The author confirmed that an accepted solution fixed the reported problem.
Closed	The discussion ended without a confirmed fix, or was closed by moderation.

Trust labels

Accepted means the original author says the solution worked for that case. **Upvoted** means community members found it useful. **Reviewed** can later mean a trusted subject-matter contributor checked its clarity and safety. These labels must remain distinct.

MVP and implementation plan

Build the smallest complete knowledge loop before adding AI or internal payments.

Stage	Deliverable	Exit test
1. Audit	Inspect the current repository, database, auth, branches, tests and deployment. Write a migration map before modifying tables.	The team knows what can be reused and what must be removed.
2. Data model	Create versioned Supabase migrations for domains, categories, problems, solutions, comments, votes, reputation events and reports.	RLS tests prove that private drafts and moderation data are protected.
3. Core journey	Post a problem, submit a solution, clarify, mark Testing, accept a solution and mark Solved.	Two accounts can complete the journey end to end.
4. Discovery	Category pages, tag filters, full-text search, solved filters and readable public case pages.	A visitor can find a seeded fix from a symptom phrase.
5. Reputation	Award transparent points from immutable events. Add basic abuse protection and category-specific profiles.	Reputation cannot be increased by self-voting or duplicate votes.
6. Tips	Allow a contributor to add an approved external tip URL. Show it only after useful participation or a solved case.	ProblemHunt never stores, routes or promises tip funds.
7. Pilot	Seed 20-30 anonymized real cases and invite 10-20 Cloud/DevOps and AV testers.	Users post, search or return without being individually reminded.
8. Launch	Deploy a reviewed build, configure analytics, monitoring, backups, moderation contact and problemhunt.cc.	Production journey, privacy rules and recovery are verified.

MVP exclusions

- No native wallet, escrow, marketplace fee or payout ledger.
- No direct messages, hiring marketplace, company workspaces or complex badges.
- No answer-generating AI until the search experience and content quality are proven.
- No expansion beyond the first two domains during the pilot.

Suggested product metrics

Metric	Why it matters
Solved rate	Shows whether posts lead to confirmed outcomes.
Time to first useful solution	Measures contributor responsiveness.
Search-to-useful-click rate	Shows whether the library retrieves relevant cases.
Accepted solutions reused	Measures compounding knowledge value.
Weekly returning contributors	Reveals whether reputation and helping are motivating.

Knowledge and AI roadmap

The long-term advantage is permissioned, structured evidence - not an uncited chatbot.

Data foundation

Each case should preserve: the user's symptom wording, environment, product/version, attempted steps, observation after each step, accepted fix, verification method, sources and edit history. Failed attempts are valuable when recorded carefully because they narrow future diagnoses.

AI progression

Phase	Capability	Guardrail
1. Search	Keyword, tags, filters and full-text search.	Rank solved and well-documented cases above guesses.
2. Similarity	Embeddings retrieve related symptoms with different wording.	Respect visibility and category boundaries.
3. Assistance	Suggest tags, detect duplicates and summarize long discussions.	User reviews every generated change.
4. Grounded answers	Answer questions from allowed, verified ProblemHunt cases.	Cite source cases and state uncertainty.
5. Pattern learning	Detect recurring faults by product, version, environment or symptom.	Use aggregated, permissioned data and validate patterns.
6. Specialized models	Consider ranking or fine-tuning only after a large reviewed dataset exists.	Benchmark against search/RAG before training.

Data rights and privacy

- Public content needs clear terms describing its display, indexing and permitted AI use.
- Private drafts, deleted material, credentials, customer identifiers and secrets must not enter training datasets.
- Provide reporting, correction, deletion and account export mechanisms.
- Do not claim that an AI answer is verified merely because it resembles an accepted case.
- Prefer retrieval-augmented generation with citations before custom model training.

Safety and moderation

Cloud and AV answers can cause outages, data loss or equipment damage. Future electrical or mechanical domains can involve physical danger. Add domain warnings, secret-redaction reminders, reporting, rate limits and moderator tools. High-risk instructions should require stronger review; some tasks should be redirected to qualified professionals.

Reputation principles

- Reputation is earned primarily from accepted and reviewed solutions, not raw posting volume.
- Keep reputation category-specific; expertise in Azure does not prove expertise in electrical work.
- Store reputation as auditable events so recalculation and abuse reversal are possible.
- Do not let tips buy visibility, reputation, acceptance or moderation influence.

Delivery architecture and working method

Transform safely on a feature branch, verify locally, then merge through review.

Recommended architecture

Layer	Responsibility
Web application	Responsive problem, solution, search and profile experiences.
Supabase Auth	Accounts, sessions and identity.
Supabase Postgres	Structured community content, votes, reputation events and moderation records.
Row-level security	Protect drafts, account data and moderator-only information.
Storage	Screenshots and logs with file validation, privacy controls and safe download handling.
Azure Static Web Apps	Frontend deployment, preview environment and problemhunt.cc hosting.
Server-side functions	Trusted reputation updates, moderation actions, webhook verification and later AI calls.

Branch strategy

Do not make this transformation directly on **main**. Create **feature/community-knowledge-platform**, keep database migrations reversible, and open a pull request after tests pass. Main should remain deployable while the product model is being replaced.

Recommended workflow

- **Use local Codex for implementation.** It can inspect the complete repository, run the application, execute tests, work with the Supabase CLI and use your existing Git credentials.
- **Use this chat for product management.** Review architecture decisions, user flows, scope, safety, metrics and each milestone before merging.
- **Work stage by stage.** Commit after the audit, schema migration, core journey, search, reputation and launch readiness rather than creating one enormous change.

Definition of done before merge

- The application contains no visible or active escrow, wallet, deposit or payout workflow.
- Two test users can complete the post-to-solved journey.
- Voting is unique, self-voting is prevented and reputation is reproducible from events.
- Public content is readable without exposing private drafts or account data.
- Search returns seeded solved cases by symptom, product and tag.
- Automated tests cover permissions and the main state transitions.
- README, environment template, migration instructions and deployment notes are current.

Launch hypothesis

If real users repeatedly search solved cases, publish new problems and confirm working fixes, ProblemHunt has a useful community loop. If they only read but never contribute, improve capture and incentives before investing in AI.

Prompt for the local Codex agent

Add this PDF under docs/ and begin a new local Codex chat in the repository.

Role and context: Act as the senior engineer for ProblemHunt. Read docs/ProblemHunt_Product_Transformation_Plan.pdf completely, inspect the repository and check AGENTS.md before making changes. ProblemHunt is becoming a free technical problem-solving community with the promise: Real problems. Tested solutions.

Product model: Users post structured Cloud/DevOps or Professional AV problems. Contributors submit possible solutions. The author tests them and accepts the solution that worked. Solved cases form a searchable public knowledge library. Users can upvote useful solutions. Contributors earn category-specific reputation and may add an external tip URL. ProblemHunt does not hold or route money.

First task: Do Stage 1 only. Audit the current codebase, schema, migrations, auth, tests, deployment configuration and working-tree status. Identify reusable components and every payment, wallet, escrow, proposal and transaction dependency that must be removed or transformed. Produce a concrete migration plan, proposed schema and file-by-file implementation sequence. Do not modify production data, deploy, delete files or change main during this audit.

Working rules: Create or use feature/community-knowledge-platform after confirming the tree is clean. Preserve unrelated work. Use versioned Supabase migrations and RLS tests. Keep secrets out of Git. Work one roadmap stage at a time, run relevant tests, summarize evidence, and wait for review before destructive schema changes or production deployment.

Product decisions already made

- Tagline: **Real problems. Tested solutions.**
- Initial domains: Cloud Computing and DevOps; Professional AV and Audio.
- Free participation; no escrow, wallet or required payment.
- Author acceptance and community upvotes are separate signals.
- Optional tips use external links in the MVP.
- AI must be grounded in permissioned cases, cite sources and communicate uncertainty.

Research references

<https://www.serviceinnovation.org/kcs/>

https://library.serviceinnovation.org/KCS/KCS_v6/KCS_v6_Practices_Guide/020

<https://docs.stripe.com/connect/charges>

<https://docs.stripe.com/connect/disputes>